

Master program
"Advanced Analytics for Business"

Text Analytics – Natural Language Processing



Universitatea Națională de Știință și Tehnologie Politehnica București
Facultatea de Automatică și Calculatoare



Course 4 Vector Space Models



Lecture content

- Vector space models
 - Definition.
 - Word by word design.
 - Word by document design.
 - Euclidian distance.
 - Cosine similarity.
 - Word embeddings.
 - Visualization using PCA.
- Python functions
 - Word embeddings and prediction.



Vector space models - definition

- Vector space models are to consider the relation ship between data that are represented by vectors.
- Applicability:
 - Information retrieval – search query.
 - Information filtering.
 - Machine translation.
- It is an algebraic model for representing word and text documents.
- “Distance” between 2 vectors measures the similarity between 2 words.
- Words, documents, tweets are encoded as vectors.
- In practice a word can be represented by a n-length vector and each element/term in the vector represents a weight of a feature that best describe/represent that word.
- **Word embeddings** – a representation for words that capture their meanings, semantic relationship and different types of context they are used in.

Word by word design

- Word by word design refers to the number of times 2 words occur together within a certain distance k .
- For a vocabulary V of length n , we can represent a word as a vector of dimension 1 to n . Example:
 - *Corpus:*
 - *I like data management.*
 - *I prefer raw data for data management.*
 - $K=2$

	like	management	raw	prefer	I
data	1	2	1	1	1

- *Capture how many times each word shows next each other word in a corpus.*

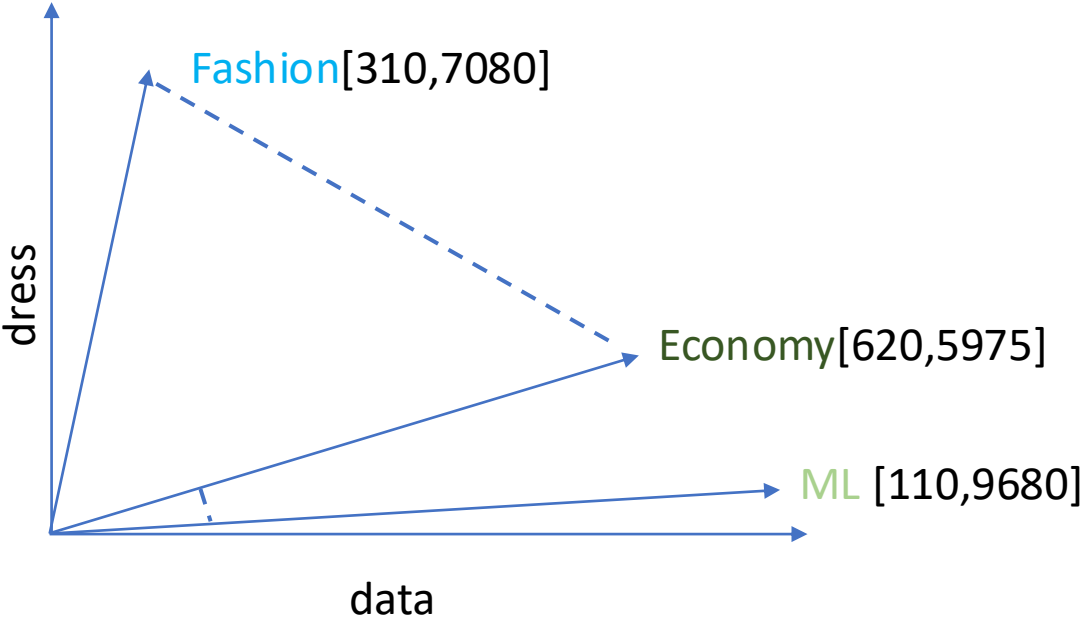
Word by document design

- Word by document design applies same concept as w-by-w and maps words inside documents.
- Point out the relation of document type and the meaning of the word.
- Considering a corpus (text, document) word by document design it is a matrix representation having as rows specific words and as columns the document category. Example:

	fashion	machine learning	economy
data	108	9680	5975
dress	7080	110	620

Word by document design - similarity

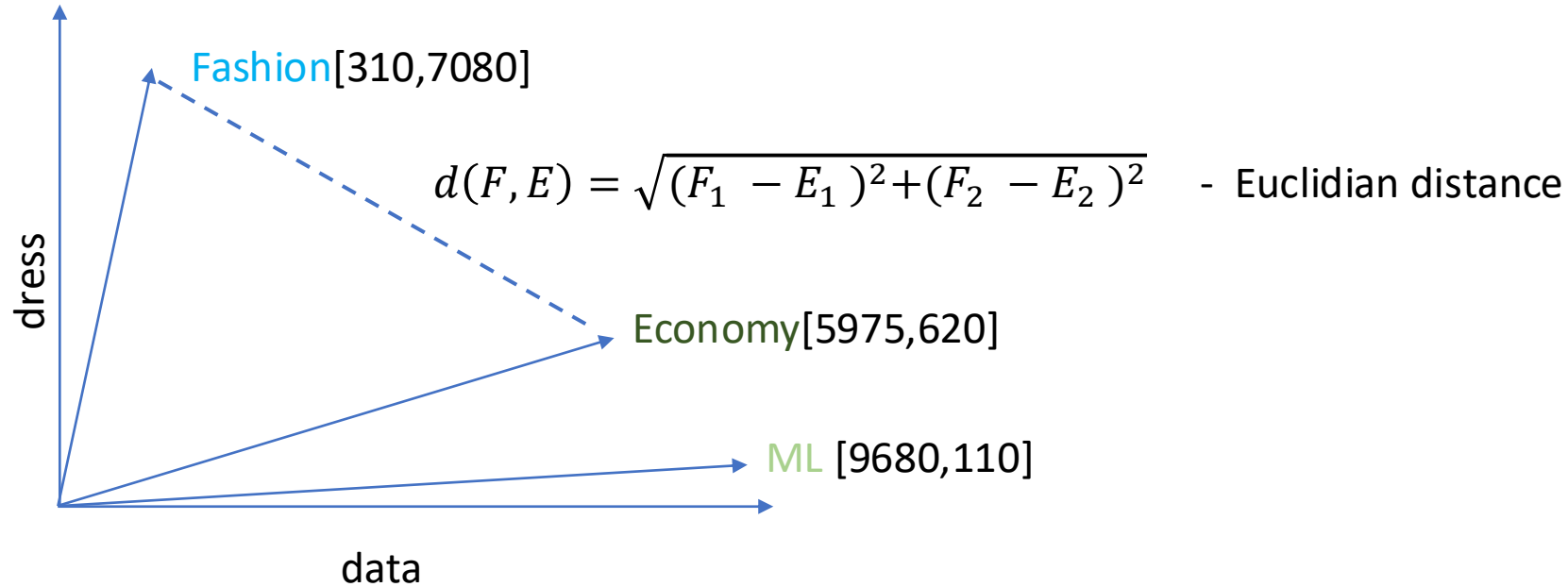
	fashion	machine learning	economy
data	310	9680	5975
dress	7080	110	620



Measures of similarity:

- Distance
- Angles

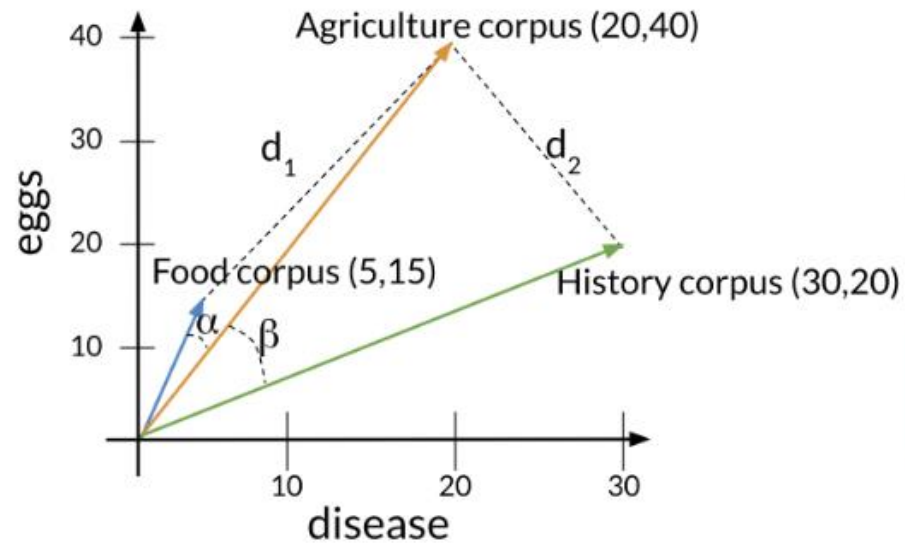
Euclidian distance



- We can generalize finding the distance between 2 points A & B to the distance between an n dimensional vector:

$$d(\vec{v}, \vec{w}) = \sqrt{\sum_{i=1}^n (v_i - w_i)^2}$$

One of the main challenge with Euclidian distance as measure of similarity is that it is not accurate and sometimes misleading. Example below:



Euclidean distance: $d_2 < d_1$

Angles comparison: $\beta > \alpha$

The cosine of the angle between the vectors

In the real scenario *food* and *agriculture* are more similar. However, in above representation distance between *food* and *agriculture* is greater than distance between *agriculture* and *history*, hence the need of introducing cosine measure.

DeepLearning.AI

Cosine similarity algebra

We introduce definitions like:

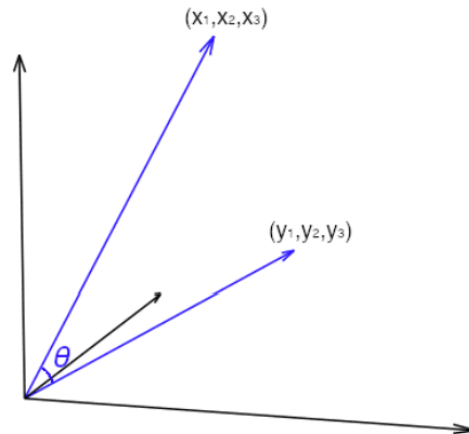
- Norm (L2):

$$\|\vec{v}\| = \sqrt{\sum_{i=1}^n |v_i|^2}$$

- Dot product:

$$\vec{v} \cdot \vec{w} = \sum_{i=1}^n v_i \cdot w_i$$

If we consider the vectors x and y in a 3D coordinate system:



$$\cos\theta = \frac{\vec{x} \cdot \vec{y}}{\|\vec{x}\| \|\vec{y}\|}$$

DeepLearning.AI

Word embeddings

- Are a type of word representation that allows words with similar meaning to have a similar representation. (*Machine Learning Mastery)
- Are a class of techniques where individual words are represented as vectors, which values are learned in a way that resembles a neural network (deep learning)
- Types of algorithms used to learn a word embedding from text data:
 - **Embedded layer:** instead of representing words as one-hot-encoded a solution is to represent into a lower-dimensional continuous vector space (100-300). In this representation we can capture intricate relationships between words – are trained using neural networks.
 - **Word2Vec:** statistical method for efficiently learning a standalone word embedding from a text corpus.
 - Types:
 - Continuous Bag-of-Words (CBOW) – learns the embedding by predicting the current word based on its context (neighboring words).
 - Continuous Skip-Gram Model – learns by predicting the surrounding words given a current word.
 - **GloVe:** The Global Vectors for Word Representation – constructs an explicit word-context or word co-occurrence matrix using statistics across the whole text corpus.

Word embeddings example

- Google Word2Vec Embedding – a pre-trained Word2Vec model from Google.
- A pre-trained model is nothing more than a file containing tokens and their associated word vectors.
- Google Word2Vec model was trained on Google news data (aprox. 100 billion words). Contains 3 million words and phrases and was fit using 300 features. Can be downloaded from:

<https://drive.google.com/file/d/0B7XkCwpl5KDYNINUTTISS21pQmM/edit?usp=sharing>

- Example:

```
from gensim.models import KeyedVectors
# load the google word2vec model
filename = 'GoogleNews-vectors-negative300.bin'
model = KeyedVectors.load_word2vec_format(filename, binary=True)
# calculate: (king - man) + woman = ?
result = model.most_similar(positive=['woman', 'son'], negative=['man'], topn=1)
print(result)
```

Dimensionality reduction

- PCA is a method for reducing the dimensionality of data. It can be used as a projection method where data with m -columns (features) is projected into a subspace with m or fewer columns, while keeping the essence of the original data.
- Algorithm:
 - Consider A ($n \times m$) matrix.
 - Calculate the mean values of each column (feature).
$$M = \text{mean}(A)$$
 - Get the centered values by subtracting the mean
$$C = A - M$$
 - Calculate the covariance matrix of matrix C (covariance of a given matrix with covariance scores for every column with every other column)
$$V = \text{cov}(C)$$
 - Calculate the eigen decomposition of the covariance matrix V
$$\text{values, vectors} = \text{eig}(V)$$

Eigenvectors represents the directions for the reduced subspace
Eigenvalues represents the magnitudes for the directions

Python functions

- Generating word embeddings (from Google Word2Vec)

```
import nltk
from gensim.models import KeyedVectors

embeddings = KeyedVectors.load_word2vec_format('./GoogleNews-vectors-negative300.bin', binary = True)
f = open('capitals.txt', 'r').read()
set_words = set(nltk.word_tokenize(f))
select_words = words = ['king', 'queen', 'oil', 'gas', 'happy', 'sad', 'city', 'town', 'village', 'country', 'continent',
                        'petroleum', 'joyful']

for w in select_words:
    set_words.add(w)

def get_word_embeddings(embeddings):

    word_embeddings = {}
    for word in embeddings.vocab:
        if word in set_words:
            word_embeddings[word] = embeddings[word]
    return word_embeddings

word_embeddings = get_word_embeddings(embeddings)
print(len(word_embeddings))
pickle.dump( word_embeddings, open( "word_embeddings_subset.p", "wb" ) )
```



Python functions

- Calculate cosine similarity

```
def cosine_similarity(A, B):  
    """  
    Inputs:  
        A: a numpy array which corresponds to a word vector  
        B: A numpy array which corresponds to a word vector  
    Output:  
        cos: numerical number representing the cosine similarity between A and B.  
    """  
    dot = ..  
    norma = ..  
    normb = ..  
    cos = ..  
  
    return cos
```



Python functions

- Euclidian distance

```
def euclidean(A, B):  
    """  
    Inputs:  
        A: a numpy array which corresponds to a word vector  
        B: A numpy array which corresponds to a word vector  
    Output:  
        d: numerical number representing the Euclidean distance between A and B.  
    """  
  
    # euclidean distance  
    d = np.linalg.norm(A-B)  
  
    return d
```

Python functions

- Simple example of predicting words relation

```
from gensim.models import KeyedVectors
# load the google word2vec model
filename = 'GoogleNews-vectors-negative300.bin'
model = KeyedVectors.load_word2vec_format(filename, binary=True)
# calculate: (woman - son) + man = ?
result = model.most_similar(positive=['woman', 'son'], negative=['man'], topn=1)
print(result)
```

```
[('daughter', 0.8796941637992859)]
```

Python functions

- Laboratory work:
 - Using cosine similarity create one function to predict country of a given capital city.
 - Steps:
 - Create one function to generate word_embeddings –see slide 13.
 - Inputs: city1, country1, city2
 - Outputs: country 2 (which has city 2 as capital)
 - Extract embeddings vectors for city1, country1, city 2
 - Get the $vec = \text{country1_emb} - \text{city1_emb} + \text{city2_emb}$
 - Create one loop to go through all word in embeddings and find the one with the highest cosine similarity comparing with vec .
 - Define the accuracy function to test the prediction performance

Python functions

- PCA decomposition – direct implementation

```
# principal component analysis
from numpy import array
from numpy import mean
from numpy import cov
from numpy.linalg import eig
A = array([
    [1, 4],
    [3, 6],
    [8, 9]])
print(A)

M = mean(A.T, axis=1)
# center columns by subtracting column means
C = A - M
# calculate covariance matrix of centered matrix
V = cov(C.T)
# factorize covariance matrix
values, vectors = eig(V)
print(vectors)
print(values)
# project data
P = vectors.T.dot(C.T)
print(P.T)
```

Python functions

- PCA decomposition – using scikit-learn library

```
# principal component analysis with scikit-learn
from numpy import array
from sklearn.decomposition import PCA

A = array([
    [1, 4],
    [3, 6],
    [8, 9]])
print(A)
# create the transform
pca = PCA(2)
# fit transform
pca.fit(A)
# access values and vectors
print(pca.components_)
print(pca.explained_variance_)
# transform data
B = pca.transform(A)
print(B)
```



Python functions

- Example of plotting using PCA:
 - We will use *gensim* library for PCA decomposition.
 - From *gensim* we will use *Word2Vec* model.
 - We will use *matplotlib* library for plotting.
 - Define sentences that will define the corpus.
 - Train a Word2Vec model based on defined sentences/corpus.
 - Dimensionality reduction using PCA with 2 elements.
 - Plot words embeddings.

Python functions

```
from gensim.models import Word2Vec
from sklearn.decomposition import PCA
from matplotlib import pyplot
# define training data
sentences = [['this', 'is', 'the', 'first', 'sentence', 'for', 'word2vec'],
              ['in', 'this', 'exercise', 'we', 'want', 'to', 'test', 'word', 'embeddings', 'using', 'NLP'],
              ['the', 'source', 'code', 'is', 'written', 'in', 'Python'],
              ['yet', 'another', 'sentence', 'programming', 'language'],
              ['one', 'more', 'sentence', 'c++']]
# train model
model = Word2Vec(sentences, min_count=1)
# fit a 2d PCA model to the vectors
X = np.array([model.wv[word] for word in model.wv.key_to_index.keys()])
pca = PCA(n_components=2)
result = pca.fit_transform(X)
# create a scatter plot of the projection
pyplot.scatter(result[:, 0], result[:, 1])
words = list(model.wv.key_to_index.keys())
for i, word in enumerate(words):
    pyplot.annotate(word, xy=(result[i, 0], result[i, 1]))
pyplot.show()
```

Python functions

